Tutorial — Pipeline de Dados NASA DONKI/CME

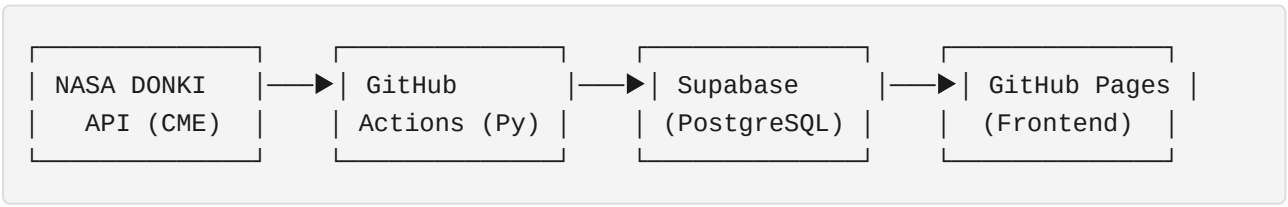
Projeto: DONKI-CME (Grupo 03 — SN-2026-GRUPO-03-NASA)
API: NASA DONKI / CME — `api.nasa.gov/DONKI/CME`
Versão: 1.0 — Outubro de 2026

Sumário

1. Visão geral e arquitetura
2. Pré-requisitos
3. Estrutura do repositório
4. Configuração do Supabase
5. Configuração do GitHub (Secrets, Pages, Actions)
6. Pipeline de coleta (`scripts/fetch_nasa.py`)
7. Painel web (`index.html`)
8. Execução local
9. Solução de problemas
10. Checklist de entrega

1. Visão geral e arquitetura

O projeto implementa um pipeline completo de dados científicos: a API DONKI da NASA fornece eventos de Ejeção de Massa Coronal (CME); o GitHub Actions executa periodicamente um script Python que coleta, normaliza e envia os dados ao Supabase (PostgreSQL); o painel estático publicado no GitHub Pages consome a API REST do Supabase e exibe tabelas, gráficos e métricas.



2. Pré-requisitos

- Conta no GitHub e acesso à organização do grupo.
- Conta no Supabase (plano gratuito) com projeto na região South America (São Paulo).
- Chave gratuita da NASA em `api.nasa.gov` (Generate API Key) — recomendada em vez de `DEMO_KEY` .
- Python 3.11+ e Node.js (opcional, para servidor local).

3. Estrutura do repositório

```
DONKI-CME/
├── index.html
├── style.css
├── script.js
├── scripts/fetch_nasa.py
├── sql/setup.sql
├── supabase/migrations/001_create_tables.sql
├── .github/workflows/update-data.yml
└── docs/
    ├── tutorial.pdf
    ├── ai-interaction.md
    ├── reflexao.md
    └── apresentacao.pdf
```

4. Configuração do Supabase

1. Crie a organização e o projeto `DONKI-CME-DB` no Supabase.
2. Abra o **SQL Editor** e execute o arquivo `sql/setup.sql`.
3. Verifique se as tabelas `cme_events` e `execucoes` foram criadas com RLS ativo.
4. Convide `rafael.ferques@ifpr.edu.br` com permissão **Developer**.

> **Constraint de deduplicação:** a coluna `activity_id` possui `UNIQUE`, permitindo upsert sem duplicatas.

> **GRANTs:** `SELECT` para `anon` e `authenticated`; `ALL` para `service_role`. As sequences também recebem `USAGE, SELECT` para `service_role`.

5. Configuração do GitHub

5.1 Organização e repositório

- Organização: `SN-2026-GRUPO-03-NASA`.
- Repositório público: `DONKI-CME`.
- Convide o usuário `GilFerques` para a organização.

5.2 Secrets (Settings → Secrets and variables → Actions)

Nome	Tipo	Descrição
SUPABASE_URL	Secret	URL do projeto Supabase
SUPABASE_SERVICE_KEY	Secret	Chave service_role (nunca no código)
NASA_API_KEY	Secret	Chave da API da NASA

5.3 GitHub Pages

Settings → Pages → Source: Deploy from a branch → branch `main`, pasta `/ (root)`.

5.4 Workflow

O arquivo `.github/workflows/update-data.yml` executa o pipeline via cron diário (06:00 UTC) e `workflow_dispatch` (manual). Boas práticas: `permissions: contents: read`, `concurrency` e `timeout-minutes: 10`.

6. Pipeline de coleta (`scripts/fetch_nasa.py`)

O script realiza as seguintes etapas:

1. Lê as variáveis de ambiente `SUPABASE_URL` , `SUPABASE_SERVICE_KEY` e `NASA_API_KEY` .
2. Consulta a API DONKI/CME no intervalo dos últimos 30 dias.
3. Normaliza os campos (incluindo a melhor análise CME e heurística de direcionamento à Terra).
4. Remove duplicatas por `activity_id` antes do upsert (evita erro PostgreSQL 21000).
5. Envia os dados em lotes de 50 registros (upsert por `activity_id`).
6. Registra a execução na tabela `execucoes` com status `concluido` , `erro_parcial` ou `erro_critico` .
7. Em caso de falha parcial/crítica, encerra com `sys.exit(1)` para visibilidade no Actions.

> A chave `service_role` nunca deve aparecer no `index.html` , em commits ou prints.

7. Painel web (`index.html`)

- Consome a API REST do Supabase com a chave pública `anon` .
- Exibe cards de métricas, gráfico de velocidade e log de eventos.
- Filtro interativo por intervalo de datas (além de presets 7D/30D/90D/1A).
- Mostra a última atualização do pipeline (lida da tabela `execucoes`).
- Aplica `escapeHtml()` a todos os valores externos inseridos no HTML.

Configuração no início de `script.js` :

```
const SUPABASE_URL = 'https://SEU_PROJETO.supabase.co';
const SUPABASE_ANON_KEY = 'SUA_CHAVE_ANON_PUBLICA';
```

8. Execução local

```
git clone https://github.com/nasa-cme/Nasa-Cme-Dashboard.git
cd Nasa-Cme-Dashboard
python -m http.server 8000
# Acesse http://localhost:8000
```

Para executar o pipeline localmente:

```
SUPABASE_URL=... SUPABASE_SERVICE_KEY=... NASA_API_KEY=... python
scripts/fetch_nasa.py
```

9. Solução de problemas

- **503 na API NASA:** a API DONKI pode estar temporariamente indisponível; reexecute o workflow mais tarde.

- **401/42501 no Supabase:** verifique se a chave usada é a `service_role` e se o RLS permite insert para `service_role`.
- **Erro PostgreSQL 21000:** garanta a deduplicação por `activity_id` antes do upsert.
- **Painel não carrega dados:** confira `SUPABASE_URL` e `SUPABASE_ANON_KEY` em `script.js` e as políticas de leitura pública.

10. Checklist de entrega

#	Item	Status
1	Nomenclaturas corretas (org, repo, projeto)	Pendente (renomear)
2	Professor adicionado no GitHub e Supabase	Pendente
3	`sql/setup.sql` com UNIQUE, RLS e GRANTs	Concluído
4	Secrets configurados	Concluído (em uso)
5	Workflow funcional (permissions, concurrency, timeout)	Concluído
6	Dados reais na tabela e em `execucoes`	Concluído (127 eventos)
7	Painel no GitHub Pages com filtro e última atualização	Concluído
8	`docs/tutorial.pdf` e `docs/apresentacao.pdf`	Concluído
9	Planilha de reserva atualizada (colunas G e H)	Pendente (manual)

Documento gerado como guia inicial do projeto. Complemente com os passos específicos do grupo.